VIT-AP UNIVERSITY · SCHOOL OF COMPUTER SCIENCE & ENGINEERING

PROJECT-BASED COMPONENT (PBC) CAPSTONE PRESENTATION

30-Minute Team Presentation Script & Evaluator Defense Guide

Project Title: Hybrid Cloud and Distributed Ledger E-Voting System

Guide: Dr. Nagendra Panini Challa | Academic Year 2025–2026

1. 30-Minute Time Budget & Role Division

Speaker	Student Details	Core Technical Domain	Time Allotted
Speaker 1	Student-1 (22BCE8319 / 24MIC7027)	Introduction, Problem Statement, Motivation & Literature Review	00:00 – 07:00 (7 min)
Speaker 2	Student-2 (22BEC8319 / 24MIC7307)	System Architecture, Hedera Hashgraph aBFT & Technology Stack	07:00 – 14:00 (7 min)
Speaker 3	Student-3 (22BCE8506 / 24MIC7302)	Concurrency Control, Atomic Claim Algorithm & Live System Demo	14:00 – 22:00 (8 min)
Speaker 4	Student-4 (22BCE8947 / 24MIC7348)	Benchmarks, STRIDE Threat Modeling, Limitations & Roadmap	22:00 – 29:00 (7 min)
All Team	Entire 4-Person Panel	Team Conclusion, Acknowledgements & Panel Defense Q&A	29:00 – 30:00+

2. Presentation Delivery & Stage Presence Guidelines

• Pacing: Maintain a measured speaking speed of ~130 words per minute. Do not rush technical figures.

• Eye Contact: Direct your attention toward Dr. Nagendra Panini Challa and the evaluation jury.

• Transitions: Explicitly hand over to the next speaker by name and summarize what they will demonstrate.

• Code & Demo: Point physically to the projector screen when highlighting key lines like findOneAndUpdate.

SPEAKER 1: INTRODUCTION, PROBLEM DOMAIN & LITERATURE REVIEW

Time Budget: 00:00 – 07:00 (7 Minutes) | Target Word Count: ~850 words

Speaker 1 (Student-1) — 00:00 to 02:00

Slide 1 & 2: Title Slide, Formal Greeting & The Centralized Database Trust Dilemma

[Stage Cue: Stand tall at center stage. Greet Dr. Nagendra Panini Challa and the panel with confident eye contact.]

"A very pleasant morning to our respected guide, Dr. Nagendra Panini Challa, and to the honorable panel of evaluators. Today, my teammates and I are proud to present our Project-Based Component capstone project: 'Hybrid Cloud and Distributed Ledger E-Voting System: A Blockchain-Based Secure Electronic Voting Platform using the Hedera Consensus Service'. My name is [Your Name], and alongside my teammates [Speaker 2], [Speaker 3], and [Speaker 4], we will walk you through the motivation, architectural design, implementation, and rigorous security benchmarking of our platform."

"Democratic elections are the cornerstone of institutional governance. Whether in collegiate councils, corporate shareholder voting, or municipal balloting, voters must be guaranteed that their ballots cannot be forged, manipulated, or erased. Over the past decade, institutions transitioned from physical paper to electronic web portals. While electronic voting accelerated counting, it created a catastrophic architectural vulnerability: Single-Point Centralized Authority."

"In conventional architectures, ballots sit in a standard relational or document database. The database administrator, hosting provider, or an attacker who compromises root credentials possesses unrestricted write access. A single SQL UPDATE statement can alter thousands of votes, and server audit logs can be cleared without leaving a tamper-evident trace."

Speaker 1 (Student-1) — 02:00 to 04:30

Slide 3 & 4: Why Pure Web3 Fails in Practice & Introducing The Hybrid Web2.5 Paradigm

[Stage Cue: Use contrasting hand gestures: left hand for Web2 flaws, right hand for Web3 friction, bringing hands together for Web2.5.]

"Naturally, computer scientists looked toward decentralized blockchains like Ethereum to achieve immutability. But deploying purely decentralized Web3 dApps for general elections creates insurmountable friction in practice."

"First, Wallet Friction: Every student or faculty member must install browser extensions like MetaMask and manage 12-word cryptographic seed phrases. If a voter loses their key, they are permanently disenfranchised. Second, Gas Volatility: Ethereum Layer-1 transactions take up to 15 minutes to confirm, and transaction gas fees fluctuate wildly from $5 to over $50 during peak congestion. It is economically absurd to expect students to purchase cryptocurrency simply to vote in a campus election. Third, Throughput Bottlenecks: Ethereum processes only 15 to 30 transactions per second, failing completely during burst voting periods."

"Our solution is an intentional Hybrid Web2.5 Architecture. On the front end, voters enjoy a standard, responsive web experience. They log in with their institutional student ID, review candidate manifestos, and submit ballots—with zero browser wallets and zero transaction fees. On the back end, our server

delegates ledger interaction to an authorized operator-relayer key that anchors anonymized ballots into the Hedera Consensus Service. Auditors can verify tallies directly from public mirror nodes without trusting our server."

Speaker 1 (Student-1) — 04:30 to 07:00

Slide 5: Comparative Architectural Analysis & Formal Handover to Speaker 2

[Stage Cue: Point to Table 2.1 on the slide. Turn to Speaker 2, gesture open-handed, and step smoothly to the side.]

"To prove our design superiority, we benchmarked our architecture against existing paradigms across five critical dimensions: Paper Ballots offer physical privacy but suffer from manual recounts taking days at costs exceeding $5 per ballot. Centralized SQL databases offer high throughput but zero public auditability. Ethereum Layer-1 provides public auditing but suffers from prohibitive gas fees and 15-minute finality. Hyperledger Fabric delivers throughput but confines auditability to consortium members."

"In stark contrast, our platform on Hedera Consensus Service delivers over 10,000 TPS throughput, irreversible consensus finality within 3 to 5 seconds, an exact predictable micro-fee of $0.0001 USD per vote, and 100% public verifiability on the HashScan block explorer."

"With this foundation laid, I now invite my colleague [Speaker 2's Name] to take you through our multi-tier system architecture and the mathematics of the Swirlds Hashgraph consensus algorithm."

SPEAKER 2: SYSTEM ARCHITECTURE, METHODOLOGY & TECH STACK

Time Budget: 07:00 – 14:00 (7 Minutes) | Target Word Count: ~850 words

Speaker 2 (Student-2) — 07:00 to 09:30

Slide 6 & 7: Multi-Tier Decoupled Architecture & End-to-End Operational Lifecycle

[Stage Cue: Step forward confidently. Direct the jury's attention to the 4 tiers in Figure 3.1 on screen.]

"Thank you, [Speaker 1]. Good morning, respected evaluators. As illustrated in Figure 3.1, our platform separates concerns across four strictly decoupled architectural tiers: The Presentation Tier is a single-page React 18 application built with Vite, providing real-time election status and candidate manifesto browsing. The Application Relayer Tier is an Express and Node.js backend executing stateless JWT authentication, role guards, and cryptographic transaction dispatch. The State Caching Tier is a local MongoDB 8 instance with WiredTiger document-level locking, acting as our first line of defense against double-voting. Finally, the Distributed Consensus Tier is the Hedera Consensus Service."

"Looking at Figure 3.2, let us trace a ballot's operational lifecycle: First, the student logs in and receives an HMAC-SHA256 signed JWT. Second, the voter selects a candidate. Third, the Express server verifies the Bearer token. Fourth, MongoDB executes an atomic claim. If the voter has already voted, the request aborts instantly with HTTP 403. Fifth, the relayer constructs an anonymized ballot payload. Sixth, the relayer signs a TopicMessageSubmitTransaction and dispatches it to Hedera Testnet Topic 0.0.10697996. Seventh, Hashgraph nodes reach consensus in 3.8 seconds. Eighth, a cryptographic transaction receipt is returned to the student."

Speaker 2 (Student-2) — 09:30 to 12:00

Slide 8: Hedera Hashgraph Consensus: Gossip about Gossip, Virtual Voting & aBFT Proofs

[Stage Cue: Emphasize the mathematical beauty of Virtual Voting: 'Nodes calculate votes in local memory without network traffic'.]

"A natural technical question is: Why did we select Hedera Hashgraph instead of a standard blockchain? Standard blockchains bundle transactions into linear blocks. If two blocks are mined simultaneously, a fork occurs and one block must be pruned, wasting network resources."

"Hedera Hashgraph departs completely from linear chains, organizing consensus events into a Directed Acyclic Graph (DAG) using two core protocols: First, Gossip about Gossip. When node A talks to node B, it transmits transaction data plus the cryptographic hashes of the two most recent events it knows. This minimal 64-byte overhead allows every node to reconstruct a complete visual graph of how information propagated across the planet."

"Second, Virtual Voting. Because every consensus node holds the complete topological event DAG, a node calculates how other nodes would vote on transaction ordering without sending vote messages across the wire. Virtual voting is mathematical, instantaneous, and generates zero network message overhead. Hashgraph is mathematically proven to be Asynchronous Byzantine Fault Tolerant (aBFT)—the highest distributed systems security standard—converging safely even under arbitrary network partitions."

Speaker 2 (Student-2) — 12:00 to 14:00

Slide 9: Containerized Tech Stack & Handover to Speaker 3 for Live Demonstration

[Stage Cue: Point to the Docker Compose architecture on slide 9. Hand over to Speaker 3.]

"To guarantee turnkey portability, our architecture is orchestrated via Docker Compose: a voting-mongodb container executes MongoDB 8 with persistent storage and healthchecks; a voting-backend container runs our Express relayer on port 5000; and a voting-frontend container serves the compiled React SPA on port 5173."

"Structured logging pipelines capture API request-response latencies, while Hedera mirror nodes provide permanent transaction archives."

"Now, to demonstrate the code that enforces concurrency control and to present our live production platform in action, I hand over the presentation to [Speaker 3's Name]."

SPEAKER 3: IMPLEMENTATION MECHANICS & LIVE SYSTEM DEMONSTRATION

Time Budget: 14:00 – 22:00 (8 Minutes) | Target Word Count: ~950 words

Speaker 3 (Student-3) — 14:00 to 16:30

Slide 10 & 11: Concurrency Control: Atomic Ballot Claim & Ledger Rollback Algorithm

[Stage Cue: Point directly to Listing 4.1 on the projector. Highlight the findOneAndUpdate query filter and the catch rollback.]

"Thank you, [Speaker 2]. Respected evaluators, in electronic voting, the most dangerous vulnerability is a race condition. If a voter fires parallel requests from multiple browser tabs, a standard read-then-write check creates a race window where both requests read hasVoted as false, resulting in duplicate ballots recorded on the ledger."

"We solved this through our Atomic Ballot Claim & Ledger Rollback Algorithm in backend/server.js. Looking at Listing 4.1 on the screen:"

"const voter = await Voter.findOneAndUpdate({ voterId: req.voterId, hasVoted: false }, { hasVoted: true, votedAt: new Date() }, { returnDocument: 'after' });"

"Under MongoDB's WiredTiger storage engine, document-level locking ensures that only ONE concurrent thread can match hasVoted: false and mutate it to true. The competing thread receives null and is instantly rejected with HTTP 403 Forbidden. Furthermore, look at our catch block: if Hedera experiences a network drop, the catch handler immediately executes an atomic rollback setting hasVoted back to false, preventing voter disenfranchisement."

Speaker 3 (Student-3) — 16:30 to 22:00

Slide 12: LIVE DEMONSTRATION: Authentication, Ballot Casting, HashScan & Mirror Node

[Stage Cue: Execute live demo on screen. Keep clicks smooth: Login -> Dashboard -> Vote Modal -> HashScan tab -> Mirror Node tab.]

"I will now switch to our live environment to demonstrate the complete workflow. First, Authentication: here is our dual-role login screen. I click the Voter Demo button to auto-fill VOTER001 and submit. The backend validates the bcrypt hash and returns a signed JWT."

"Second, Voter Dashboard & Manifestos: the dashboard displays active candidates. Notice our Manifesto tracking feature: voters can review Sarah Jenkins's campaign pledges and examine historical promises vs performance."

"Third, Cryptographic Ballot Submission: I select Sarah Jenkins and click Vote. Upon confirming, our backend atomically claims the voter record and dispatches a TopicMessageSubmitTransaction to Hedera Testnet Topic 0.0.10697996."

"Fourth, Consensus Finality: in under 4 seconds, consensus is reached! The receipt is displayed with our unique Transaction ID: 0.0.9757391@1730182450.123456789."

"Fifth, Public HashScan Verification: switching to hashscan.io/testnet, we search Topic 0.0.10697996. Here is our newly minted message with monotonic Sequence Number and nanosecond timestamp! Sixth, Mirror Node Audit: querying testnet.mirrornode.hedera.com directly returns the base64 payload

containing candidateId. Any auditor can compute the election tally independently without trusting our server."

"I now invite [Speaker 4's Name] to present our benchmark results, formal threat modeling, and future roadmap."

SPEAKER 4: BENCHMARKS, SECURITY THREAT MODEL & FUTURE ROADMAP

Time Budget: 22:00 – 29:00 (7 Minutes) | Target Word Count: ~850 words

Speaker 4 (Student-4) — 22:00 to 24:30

Slide 13: Quantitative Benchmarks: Concurrency, Consensus Latency & Duplicate Rejections

[Stage Cue: Present Table 5.1 with confidence. Highlight sub-5-second finality and 100% duplicate rejection under race testing.]

"Thank you, [Speaker 3]. Respected professors, an engineering design is only as credible as its empirical data. We subjected our backend relayer to concurrent load testing across four tiers: 5, 10, 25, and 50 simultaneous voters connected to Hedera Testnet."

"As detailed in Table 5.1: At 5 and 10 concurrent voters, the system achieved a 100% success rate with an average consensus finality of 3.8 to 4.1 seconds. At 25 concurrent voters with simulated race conditions, our atomic claim successfully rejected 100% of duplicate ballot attempts with HTTP 403, guaranteeing zero double-voting on the ledger."

"Under burst load of 50 concurrent voters, we achieved a 98% first-pass completion rate with 4.8 seconds average finality. The two dropped requests were due to local test socket exhaustion, and both safely rolled back in the database and completed on retry. Across all tests, the ledger count matched our database with 100% mathematical consistency."

Speaker 4 (Student-4) — 24:30 to 27:00

Slide 14 & 15: Formal STRIDE Threat Model, OWASP Top 10 & Regression Test Verification

[Stage Cue: Walk through STRIDE systematically: Spoofing, Tampering, Repudiation, Information Disclosure, DoS, Elevation.]

"We formally evaluated the platform against Microsoft's STRIDE Threat Framework: Spoofing is countered by salted bcrypt password hashes and 24-hour expiration JWTs. Tampering is mitigated because even a compromised database administrator cannot alter votes recorded on Hedera. Repudiation is eliminated because voters receive cryptographic transaction receipts verifiable on HashScan. Information Disclosure is addressed by decoupling voter identities from on-chain payloads. Denial of Service is prevented via atomic database indexing and token rate limiting. Elevation of Privilege is blocked by requireRole middleware."

"Against OWASP Top 10 vulnerabilities, we implemented strict NoSQL input sanitization (.trim().toUpperCase()), externalized secret configurations, and HTTPS cryptographic transport. Our automated regression test suite executed with a 100% pass rate across 7 security test cases."

Speaker 4 (Student-4) — 27:00 to 29:00

Slide 16: System Limitations, Strategic Future Roadmap & Handover for Conclusion

[Stage Cue: Acknowledge limitations objectively. Detail zk-SNARKs and DIDs as clear mathematical

evolutions of the project.]

"Engineering honesty requires acknowledging limitations: First, Operator Balance Depletion: because our relayer sponsors transactions at $0.0001 each, if the account runs out of HBAR, submissions fail. Automated treasury monitors mitigate this. Second, Mirror Node Eventual Consistency: mirror nodes index transactions within 1 to 2 seconds after consensus, requiring a brief grace period before REST queries reflect new tallies."

"Our strategic roadmap includes three innovations: First, Zero-Knowledge Proofs (zk-SNARKs) to mathematically prove a voter's eligibility without disclosing their candidate choice or voter ID to the backend. Second, W3C Decentralized Identifiers (DIDs) for university-wide self-sovereign identity. Third, Multi-Signature Governance requiring threshold keys from multiple faculty trustees to initialize election topics."

"I now call upon [Speaker 1's Name] to conclude our presentation."

CONCLUSION & EVALUATOR DEFENSE CHEAT SHEET

Time Budget: 29:00 – 30:00+ | Team Delivery & Panel Q&A Defense

Speaker 1 & Team — 29:00 to 30:00

Slide 17: Project Summary, Academic Acknowledgements & Opening of Q&A

[Stage Cue: All 4 team members stand in line facing the panel with professional posture.]

"To conclude: our project proves that we do not need to choose between the usability of modern web applications and the immutable security of distributed ledgers. By synthesizing an operator-relayer pattern with the Hedera Consensus Service, we have delivered an electronic voting system that is 100% immune to database manipulation, free of crypto wallet friction for voters, benchmarked to under 5-second finality at a predictable cost of $0.0001 per vote, and fully auditable by any student on public mirror nodes."

"We express our sincere gratitude to our faculty guide, Dr. Nagendra Panini Challa, for his mentorship. We are now open for questions from the evaluation panel. Thank you!"

3. Evaluator Defense Q&A Strategy (Anticipated Panel Questions)

Q1: If your backend relayer signs all transactions with a single operator key, isn't your relayer still a centralized point of failure?

Primary Defender: Speaker 3 (or Speaker 2)

Defensive Answer Strategy:

"While the relayer sponsors gas fees, it cannot alter vote counts. If the relayer attempts to forge a vote, the raw message payload recorded on Hedera's public mirror nodes will not match the voter's receipt. Because Hedera assigns strictly increasing sequence numbers, the backend cannot secretly insert or delete votes without causing a visible sequence break. On mainnet, we replace the single operator key with a Hedera threshold multi-signature account requiring multiple faculty members to co-sign."

Q2: Explain exactly how your atomic claim in MongoDB prevents double-voting if two requests hit the server at the exact same microsecond.

Primary Defender: Speaker 3

Defensive Answer Strategy:

"We use MongoDB's atomic findOneAndUpdate operator with the filter condition { voterId: req.voterId, hasVoted: false }. Under MongoDB's WiredTiger storage engine, document-level locking is applied during writes. Even if two requests arrive at the identical millisecond on separate threads, MongoDB serializes the write on that specific document. The first thread acquires the lock and mutates hasVoted to true. The second thread evaluates the filter, finds zero matching documents, and returns null. Our code checks if (!voter) and immediately returns HTTP 403 Forbidden before any network call to Hedera is initiated."

Q3: Why did you choose Hedera Consensus Service instead of an Ethereum Layer-2 rollup like

Polygon or Arbitrum?

Primary Defender: Speaker 1

Defensive Answer Strategy:

"We evaluated Polygon and Arbitrum during our literature review. While L2 rollups reduce fees, they have two major drawbacks: First, gas volatility: even on Polygon, fees fluctuate based on rollup traffic, requiring unpredictable treasury management. Hedera fees are pegged to fiat USD and fixed at exactly $0.0001 USD per message, providing an exact budget of $1.00 per 10,000 votes. Second, finality time: Polygon requires 2 to 4 minutes for state commitment to Ethereum L1. Hedera Hashgraph achieves true aBFT finality in 3.8 seconds with zero risk of chain reorganizations."

Q4: How do you protect voter privacy? Can someone look at the public mirror node and see who a specific student voted for?

Primary Defender: Speaker 4

Defensive Answer Strategy:

"Voter privacy is protected by structural decoupling. The message payload broadcast to Hedera contains only { candidateId, timestamp }. It contains zero student names, email addresses, or voter registration IDs. An auditor on HashScan sees that a valid vote was cast for Candidate A at consensus timestamp T, but there is zero cryptographic link on the public ledger connecting that ballot to a specific student identity. For complete mathematical anonymity against a compromised backend database, our future work implements client-side zk-SNARK proofs."

Q5: What happens if a student claims they voted, but the university administration claims they didn't?

Primary Defender: Speaker 4

Defensive Answer Strategy:

"This is the Non-Repudiation guarantee of our STRIDE evaluation. When a vote succeeds, the voter's browser receives and saves a cryptographic Transaction ID. The student can search that transaction ID on hashscan.io. The public Hedera network independently proves whether that transaction was included in Topic 0.0.10697996, at what exact nanosecond, and with what sequence number. Neither the student nor the university can falsely dispute a verified on-chain consensus receipt."

4. Pre-Presentation Technical Checklist

1. Backend running: node server.js on port 5000 connected to MongoDB and Hedera Testnet.
2. Frontend running: npm run dev on port 5173 with browser opened to http://localhost:5173.
3. Browser Tab 1: Frontend Login & Voter Dashboard ready.
4. Browser Tab 2: HashScan Explorer (https://hashscan.io/testnet/topic/0.0.10697996) loaded.
5. Browser Tab 3: Mirror Node REST API endpoint pre-tested and ready.
6. Backup mobile hotspot connected in case university Wi-Fi blocks Hedera gRPC port 50211.